

东北大学资源与土木工程学院

作 业 报 告

课程名称: 雷达干涉测量

学 生 专 业: 测绘工程

学生班级: 测绘 2301 班

学 号 : 20232411

学生姓名: 张洋

指导教师姓名: 魏恋欢、敖萌、王一安

指导教师评估:

评 阅 人 :

评阅时间:

目录

一、核心任务.....	1
二、核心算法.....	1
2.1 基础数据处理与可视化.....	1
2.2 干涉图生成.....	2
2.3 相干系数计算.....	2
2.4 平地效应去除.....	2
2.4.1 平地相位模型.....	3
2.4.2 去平地相位.....	3
2.5 地形相位去除.....	3
2.5.1 高程差计算.....	3
2.5.2 地形相位模型.....	3
2.5.3 去地形相位.....	3
2.6 相位滤波算法.....	3
2.6.1 中值滤波.....	3
2.6.2 自适应滤波.....	4
2.6.3 复数维纳滤波.....	4
2.6.4 Goldstein 滤波.....	4
2.6.5 深度学习自监督滤波.....	5
三、程序流程.....	5
3.1 数据读取与预处理.....	6
3.2 基础数据可视化.....	6
3.3 干涉图与相干系数计算.....	6
3.4 平地效应去除.....	6
3.5 地形相位去除.....	6
3.6 传统滤波实现.....	7
3.7 深度学习滤波实现.....	7
3.8 结果保存与可视化.....	7

四、结果分析.....	7
4.1 基础数据可视化结果.....	7
4.1.1 DEM 影像.....	7
4.1.2 主/辅影像幅度图.....	8
4.1.3 主/辅影像相位图.....	9
4.2 干涉图结果.....	10
4.3 相干系数图结果.....	11
4.4 去平地效应结果.....	12
4.4.1 平地相位模型图.....	12
4.4.2 去平地效应后的相位图.....	13
4.5 去地形相位结果.....	14
4.5.1 地形相位模型图.....	14
4.5.2 去平地且去地形后相位图.....	14
4.6 滤波结果对比.....	15
4.6.1 中值滤波效果图.....	15
4.6.2 自适应滤波效果图.....	16
4.6.3 复数维纳滤波效果图.....	16
4.6.4 Goldstein 滤波效果图.....	17
4.6.5 深度学习自监督滤波结果.....	18
4.6.6 滤波方法综合对比.....	19
五、总结.....	19
六、个人收获与感悟.....	20
参考文献.....	21
附录 I InSAR 数据处理 Python 代码.....	22

雷达干涉测量（InSAR）数据处理与分析报告

本报告围绕雷达干涉测量（InSAR）的核心数据处理流程展开，基于给定的区域的 InSAR 数据，完成了数字高程模型（DEM）与主辅影像幅度、相位可视化、干涉图生成、相干系数计算、平地效应与地形相位去除，以及多种相位滤波方法的实现与对比分析。

一、核心任务

本次 InSAR 编程作业的核心任务主要是干涉相位数据的全流程处理与可视化分析，具体为以下五个关键环节：

1. 基础数据可视化：读取 MAT 格式的 InSAR 数据，提取数字高程模型（DEM）、主影像（ im_m ）、辅影像（ im_s ），绘制 DEM 影像、主/辅影像的幅度图与相位图，完成原始数据的特征呈现；

2. 干涉图生成：通过主辅影像的复数共轭相乘与相位提取，生成干涉相位图，直观反映两景影像的相位差异；

3. 相干系数计算：基于局部窗口的统计特性，计算主辅影像的相干系数并可视化，量化影像间的相关性；

4. 相位误差去除：构建平地相位模型与地形相位模型，依次去除干涉相位中的平地效应与地形相位，分离出地表形变相关的相位信息；

5. 相位滤波优化：实现中值滤波、自适应滤波、复数维纳滤波、Goldstein 滤波及深度学习自监督滤波五种方法，对比不同滤波策略对噪声相位的抑制效果与细节保留能力。

二、核心算法

2.1 基础数据处理与可视化

InSAR 数据中，主影像（ im_m ）与辅影像（ im_s ）均以复数形式存储，复数的模表示回波信号的幅度，辐角表示相位。对于复数影像 $I = a + bj$ ，幅度与相位的计算公式为：

$$|I| = \sqrt{a^2 + b^2} \quad (1)$$

$$\angle(I) = \arctan\left(\frac{b}{a}\right) \quad (2)$$

为提升幅度图的视觉可读性，对幅度值进行对数变换：

$$A_{dB} = 10 \log_{10}(|I|) \quad (3)$$

2.2 干涉图生成

干涉图的本质是主辅影像的相位差，通过复数共轭相乘实现相位相减，再提取辐角得到干涉相位：

$$I_{\text{conj}} = im_m \cdot \overline{im_s} \quad (4)$$

$$\phi_{\text{ifg}} = \angle(I_{\text{conj}}) \quad (5)$$

其中 $\overline{im_s}$ 表示辅影像的复数共轭， ϕ_{ifg} 为干涉相位，反映两景影像的相位差异，包含平地、地形、形变等多种信息。

2.3 相干系数计算

相干系数(γ)用于衡量主辅影像在局部窗口内的相关性，取值范围为[0,1]，越接近 1 表示相关性越强，相位噪声越低。其计算基于局部窗口的统计特性，公式如下：

$$\gamma = \frac{|\sum_{i,j \in \text{win}} im_m(i,j) \cdot \overline{im_s(i,j)}|}{\sqrt{\sum_{i,j \in \text{win}} |im_m(i,j)|^2 \cdot \sum_{i,j \in \text{win}} |im_s(i,j)|^2}} \quad (6)$$

为简化计算，采用滑动窗口的均值滤波近似实现局部求和：

$$\text{num} = \text{uniform_filter}(im_m \cdot \overline{im_s}, \text{size} = N) \quad (7)$$

$$\text{pwr}_{im_m} = \text{uniform_filter}(|im_m|^2, \text{size} = N) \quad (8)$$

$$\text{pwr}_{im_s} = \text{uniform_filter}(|im_s|^2, \text{size} = N) \quad (9)$$

$$\gamma = \frac{|\text{num}|}{\sqrt{\text{pwr}_{im_m} \cdot \text{pwr}_{im_s}}} \quad (10)$$

其中 N 为窗口大小，本次实验取 7×7 ，最后通过clip函数将相干系数限制在[0,1]范围内。

2.4 平地效应去除

平地效应是由雷达侧视成像几何导致的相位趋势，与地形无关，需通过几何建模去除。核心公式如下：

2.4.1 平地相位模型

$$\phi_{\text{flat}} = -\frac{4\pi}{\lambda} \cdot \frac{B_{\perp}}{R_0 \tan \theta} \cdot x \quad (11)$$

其中 λ 为雷达波长（本次取 0.056m）； $B_{\perp}$ 为垂直基线（本次取 95.9m）； R_0 为中心斜距（本次取 847587.2953m）； θ 为入射角（弧度制，由 36.6569° 转换）； x 为距离向像素坐标（以影像中心为原点， $x = (\text{cols} - \text{cols}/2) \cdot dr$ ， dr 为距离向像素间距，本次取 2.329541m）。

将一维平地相位扩展至二维影像维度：

$$\phi_{\text{flat-2D}} = \text{tile}(\phi_{\text{flat-1D}}, (\text{rows}, 1)) \quad (12)$$

2.4.2 去平地相位

在复平面内实现相位相减，避免相位缠绕误差：

$$I_{\text{flat-removed}} = e^{j(\phi_{\text{ifg}} - \phi_{\text{flat-2D}})} \quad (13)$$

$$\phi_{\text{flat-removed}} = \angle(I_{\text{flat-removed}}) \quad (14)$$

2.5 地形相位去除

地形相位由研究区域的高程差异导致，需基于 DEM 数据建模并去除：

2.5.1 高程差计算

以 DEM 的中位数高程为参考面，计算高程偏差：

$$h_{\text{ref}} = \text{nanmedian}(\text{dem}) \quad (15)$$

$$\Delta h = \text{dem} - h_{\text{ref}} \quad (16)$$

2.5.2 地形相位模型

$$\phi_{\text{topo}} = -\frac{4\pi}{\lambda} \cdot \frac{B_{\perp}}{R_0 \sin \theta} \cdot \Delta h \quad (17)$$

2.5.3 去地形相位

同样在复平面内完成相位相减：

$$I_{\text{topo-removed}} = e^{j(\phi_{\text{flat-removed}} - \phi_{\text{topo}})} \quad (18)$$

$$\phi_{\text{topo-removed}} = \angle(I_{\text{topo-removed}}) \quad (19)$$

2.6 相位滤波算法

2.6.1 中值滤波

中值滤波通过替换窗口内的相位中值实现噪声抑制，公式为：

$$\phi_{\text{med}} = \text{median}(\phi_{\text{in}}(i,j) | (i,j) \in \text{win}) \quad (20)$$

滤波后重新对相位进行缠绕，确保相位范围在 $[-\pi, \pi]$ ：

$$\phi_{\text{med-wrap}} = \angle(e^{j\phi_{\text{med}}}) \quad (21)$$

2.6.2 自适应滤波

基于局部复相位一致性（ ρ ）构建自适应权重，实现高一致性保细节、低一致性平滑：

1. 复数影像转换：

$$z_{\text{in}} = e^{j\phi_{\text{in}}} \quad (22)$$

2. 局部均值计算：

$$z_{\text{mean}} = \text{uniform_filter}(\text{Re}(z_{\text{in}})) + j \cdot \text{uniform_filter}(\text{Im}(z_{\text{in}})) \quad (23)$$

3. 局部一致性：

$$\rho = |z_{\text{mean}}| \quad (24)$$

4. 自适应权重：

$$\alpha = \text{clip}\left(\frac{\rho - 0.25}{0.50}, 0, 1\right) \quad (25)$$

5. 自适应滤波：

$$z_{\text{adapt}} = \alpha \cdot z_{\text{in}} + (1 - \alpha) \cdot z_{\text{mean}} \quad (26)$$

6. 相位提取：

$$\phi_{\text{adapt}} = \angle(z_{\text{adapt}}) \quad (27)$$

2.6.3 复数维纳滤波

对复数影像的实部和虚部分别进行维纳滤波，再重构复数并提取相位：

$$\text{Re}(z_{\text{wiener}}) = \text{wiener}(\text{Re}(z_{\text{in}}), \text{win} = (7,7)) \quad (28)$$

$$\text{Im}(z_{\text{wiener}}) = \text{wiener}(\text{Im}(z_{\text{in}}), \text{win} = (7,7)) \quad (29)$$

$$\phi_{\text{wiener}} = \angle(\text{Re}(z_{\text{wiener}}) + j \cdot \text{Im}(z_{\text{wiener}})) \quad (30)$$

2.6.4 Goldstein 滤波

基于频域幅度加权实现相位平滑，核心步骤：

1. 复数影像傅里叶变换：

$$S = \text{FFT2}(z_{\text{in}}) \quad (31)$$

2. 幅度提取:

$$|S| = \text{abs}(S) \quad (32)$$

3. 权重构建 ($\alpha = 0.7$ 为经验值, $\epsilon = 1e - 8$ 避免除零):

$$W = \left(\frac{|S|}{\max(|S|) + \epsilon} \right)^\alpha \quad (33)$$

4. 频域加权:

$$S_{\text{filtered}} = S \cdot W \quad (34)$$

5. 逆傅里叶变换:

$$z_{\text{goldstein}} = \text{IFFT2}(S_{\text{filtered}}) \quad (35)$$

6. 相位提取:

$$\phi_{\text{goldstein}} = \angle(z_{\text{goldstein}}) \quad (36)$$

2.6.5 深度学习自监督滤波

基于 SmallUNet 网络, 以“掩码重建”为自监督任务实现相位滤波:

1. 相位编码: 将相位转换为二维特征 (余弦+正弦):

$$f = [\cos(\phi), \sin(\phi)] \quad (37)$$

2. 数据集构建: 对特征图随机掩码 (掩码率 10%), 构建“掩码输入-原始输出”的训练对;

3. 网络结构: 采用 U-Net 架构, 包含编码 (DoubleConv+MaxPool)、瓶颈 (DoubleConv)、解码 (ConvTranspose2d+DoubleConv) 及输出层;

4. 损失函数: 掩码区域的均方误差 (MSE):

$$\text{Loss} = \text{MSE}(f_{\text{pred}} \cdot M, f_{\text{target}} \cdot M) \quad (38)$$

其中 M 为掩码矩阵;

5. 相位解码: 滤波后特征图还原为相位:

$$\phi_{\text{dl}} = \arctan 2(f_{\text{out}}[1], f_{\text{out}}[0]) \quad (39)$$

三、程序流程

本次 InSAR 数据处理的程序基于 Python 实现, 核心依赖 NumPy (数值计算)、Matplotlib (可视化)、Scipy (滤波/统计)、PyTorch (深度学习) 等库, 整体流程可分为 8 个核心步骤, 具体如下:

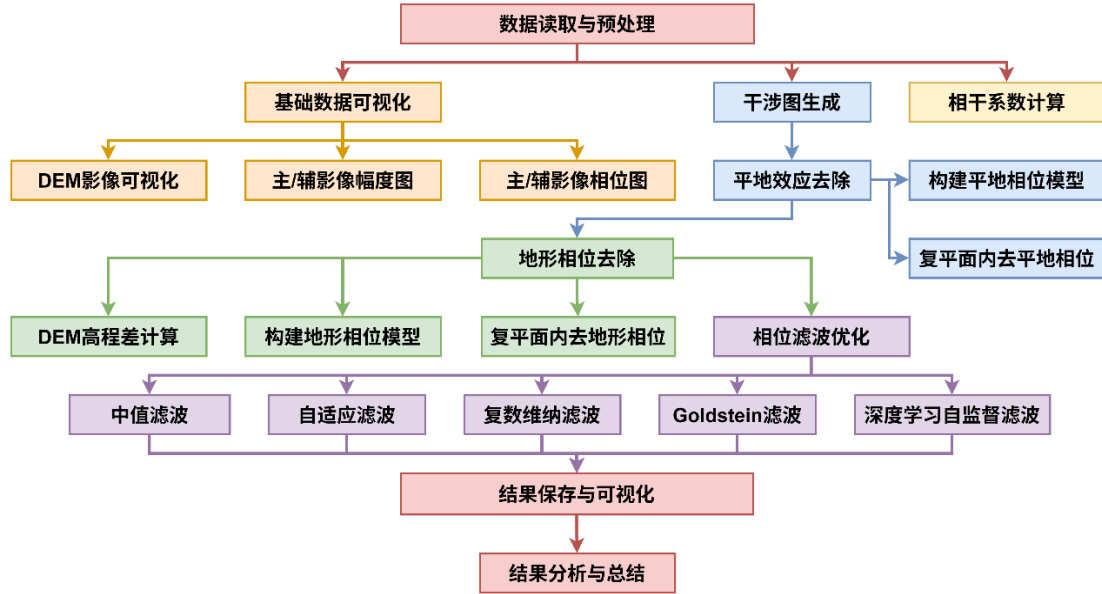


图 1 程序流程图

3.1 数据读取与预处理

1. 调用 `scipy.io.loadmat` 读取 MAT 格式的 InSAR 数据，提取 DEM (`dem`)、主影像 (`im_m`)、辅影像 (`im_s`)；
2. 将数据转换为 NumPy 数组，定义可视化函数 `showImg`（设置字体、画布大小、颜色映射、保存图片等），并设定影像纵横比 `scale=20/9`。

3.2 基础数据可视化

1. 调用 `showImg` 绘制 DEM 影像（颜色映射 `jet`）；
2. 计算主/辅影像的幅度（对数变换）与相位，分别调用 `showImg` 绘制幅度图（`gray`）与相位图（`jet`）。

3.3 干涉图与相干系数计算

1. 主辅影像共轭相乘，提取干涉相位，绘制干涉图；
2. 分别基于 3×3 、 5×5 、 7×7 滑动窗口，通过均值滤波计算局部共轭乘积、主/辅影像功率，进而计算相干系数，绘制相干系数图。

3.4 平地效应去除

1. 定义入射角、垂直基线、波长、中心斜距、距离向像素间距等几何参数；
2. 构建一维平地相位模型，扩展为二维；
3. 复平面内减去平地相位，绘制平地相位模型图与去平地效应后的相位图。

3.5 地形相位去除

1. 计算 DEM 中位数高程，构建高程差 Δh ；

2. 建模地形相位，复平面内减去地形相位；
3. 绘制地形相位模型图与去平地+去地形后的相位图。

3.6 传统滤波实现

依次实现中值滤波（ 5×5 窗口）、自适应滤波（ 7×7 窗口）、复数维纳滤波（ 7×7 窗口）、Goldstein 滤波（ $\alpha = 0.7$ ），分别绘制各滤波后的相位图。

3.7 深度学习滤波实现

1. 定义相位编码/解码函数、数据集类、SmallUNet 网络；
2. 构建数据加载器，设置训练参数（批次 4、轮次 10、学习率 $1e-3$ ）；
3. 以掩码重建为任务训练网络，训练完成后对全幅相位图推理；
4. 解码得到滤波后相位，绘制深度学习滤波结果图。

3.8 结果保存与可视化

所有步骤的可视化结果均通过 showImg 函数保存为 PNG 格式（300DPI），并实时显示。

四、结果分析

4.1 基础数据可视化结果

4.1.1 DEM 影像

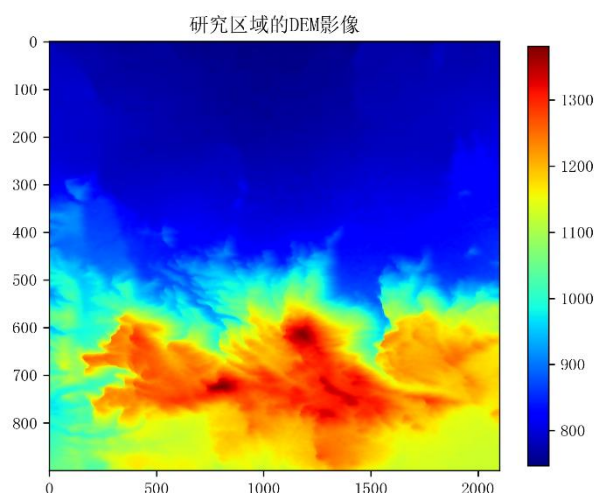


图 2 研究区域的 DEM 影像

图 2 为研究区域的数字高程模型（DEM）影像。从整体空间分布来看，研究区呈现出北低南高、阶梯式抬升的典型地形特征：影像上半部分以深蓝色为主，高程集中在 800~900m，为地势平缓的低海拔区域；下半部分高程快速抬升，以

橙红色、亮红色为主，最高高程超过 1300m，为地形起伏剧烈的高海拔山地，局部存在明显的山脊、沟谷等微地形结构，高程梯度变化显著。

本 DEM 影像为后续地形相位的精准建模与去除提供了基准，同时也直观反映了研究区的地形复杂度，为干涉相位噪声来源提供了地理层面的解释依据。

4.1.2 主/辅影像幅度图

图 3（主影像幅度图）与图 4（辅影像幅度图）为经过 $10\log_{10}$ 对数变换后的 SAR 幅度可视化结果。从整体空间分布来看，主、辅影像的幅度特征高度一致：影像下半部分呈现出清晰的山脊、沟谷等地形纹理，与 DEM 影像中高海拔、地形起伏剧烈的区域完全对应，这是由于陡峭地形的雷达入射角变化导致回波散射特性差异显著；而上半部分为低海拔平缓区域，幅度分布均匀，纹理特征不明显，符合平坦地表的散射规律。两景影像的幅度一致性验证了主辅影像的配准精度与散射体的时间稳定性，为后续干涉相位的生成提供了可靠的基础；同时，局部区域的细微幅度差异反映了两景影像成像时间差导致的地表散射特性变化，这也是后续相干系数图中低相干区域的主要来源之一。。

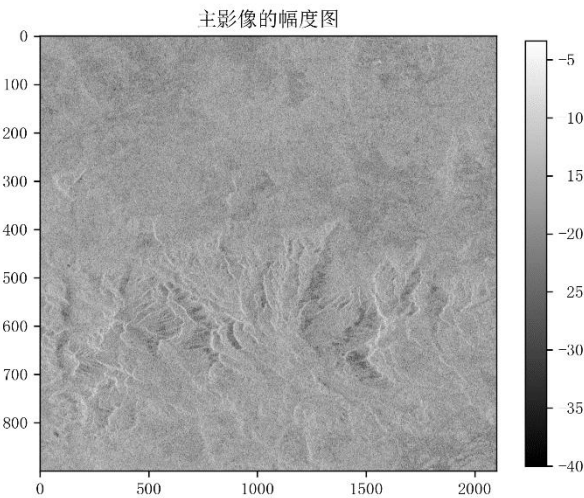


图 3 主影像的幅度图

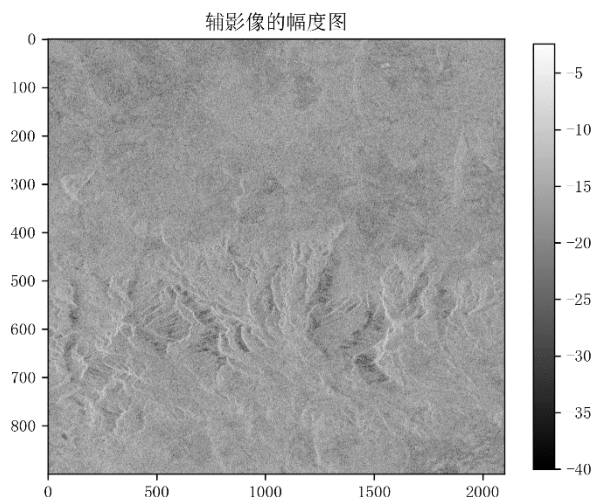


图 4 辅影像的幅度图

4.1.3 主/辅影像相位图

图 5（主影像相位图）与图 6（辅影像相位图）为 SAR 复数影像提取的原始相位可视化结果，采用 jet 色阶呈现。从整体特征来看，主、辅影像的原始相位均呈现出高噪声、随机分布的特点，相位值在 $[-\pi, \pi]$ 区间内剧烈跳变，无明显的地形或形变相关的连续纹理特征。同时，两景影像的相位分布存在高度相似的全局条纹趋势，这一相关性是干涉图生成的核心前提。此外，相位图中呈现的周期性水平条纹，本质上是雷达侧视成像几何引入的平地相位趋势，这也为后续平地效应的去除提供了直观的验证依据。

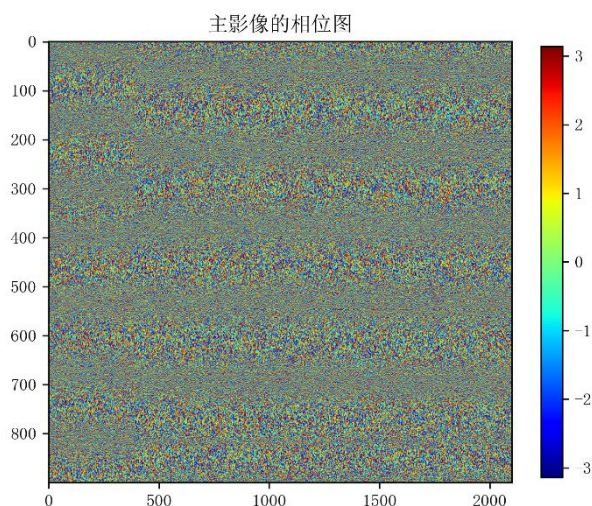


图 5 主影像的相位图

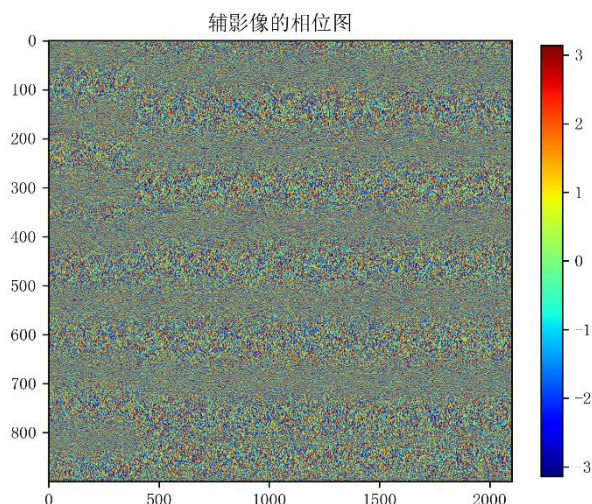


图 6 辅影像的相位图

4.2 干涉图结果

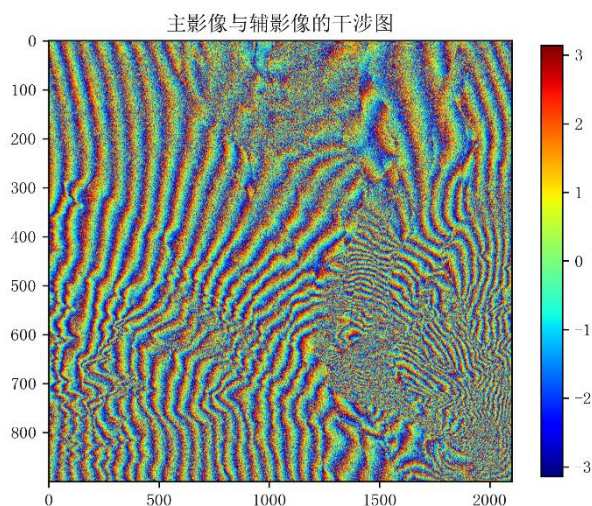


图 7 主影像与辅影像的干涉图

图 7 为主影像与辅影像经复数共轭相乘后提取的干涉相位图，其中蓝、青、黄、红的周期性循环代表相位从 $-\pi$ 到 π 的缠绕变化，每一次完整的色彩循环对应一个 2π 的相位周期。从整体特征来看，干涉图呈现出显著的斜向密集条纹，这是雷达侧视成像几何引入的平地效应主导的相位趋势，说明原始干涉相位中包含大量与地形、形变无关的系统性相位成分；同时，在影像下半部分，条纹出现明显的扭曲、变形与密度变化，这是地形高程差异引入的地形相位与平地相位叠加的结果，直观反映了地形对干涉相位的调制作用。干涉图的条纹特征验证了主辅影像的相位相关性，同时也体现了原始干涉相位的复杂性。

4.3 相干系数图结果

图 8、图 9、图 10 分别为采用 3×3 、 5×5 、 7×7 滑动窗口计算得到的主辅影像相干系数图。从空间分布来看，三张相干系数图的整体规律高度一致：影像左下半部分以橙红色为主，相干系数集中在 $0.6\sim 0.8$ ，对应 DEM 中地形起伏剧烈的高海拔山地，说明该区域散射体稳定性强、相位噪声低；而影像右上半部分以蓝青色为主，相干系数多低于 0.4 ，对应低海拔平缓区域，推测为植被覆盖区，散射体时间去相干效应显著，相位噪声水平高。

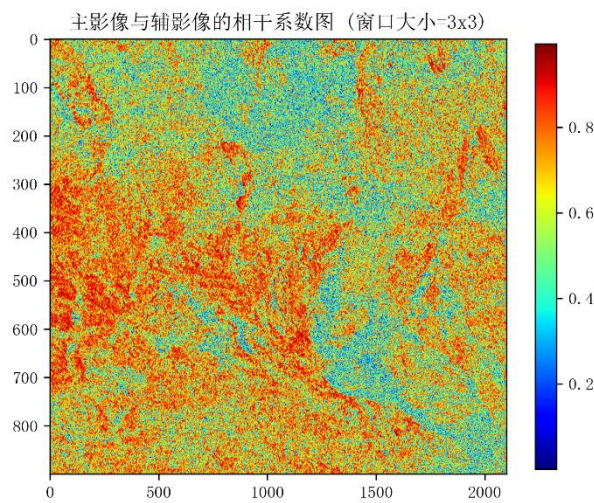


图 8 主影像与辅影像的相干系数图（窗口大小=3x3）

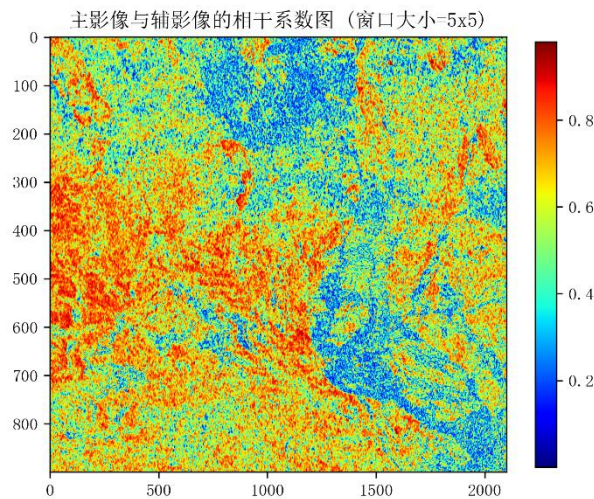


图 9 主影像与辅影像的相干系数图（窗口大小=5x5）

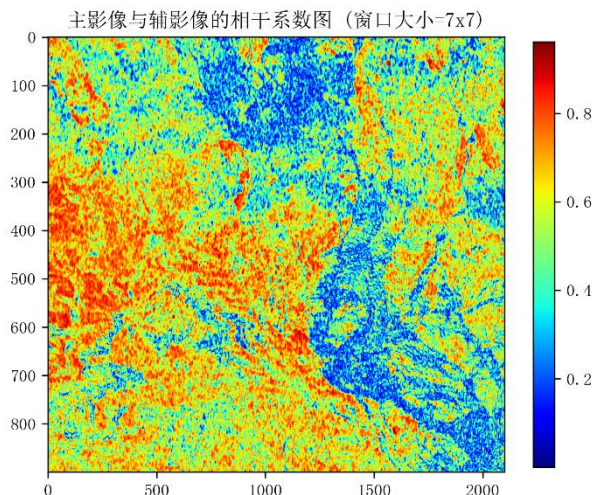


图 10 主影像与辅影像的相干系数图 (窗口大小=7x7)

对比不同窗口大小的结果可见： 3×3 窗口的相干系数图噪声水平最高，局部相干值波动剧烈，高、低相干区域的边界模糊； 5×5 窗口的噪声得到一定抑制，区域分布更清晰，但仍存在局部颗粒感； 7×7 窗口的相干系数图平滑度最优，高、低相干区域的纹理特征完整保留，在噪声抑制与细节保留间实现了最佳平衡。

4.4 去平地效应结果

4.4.1 平地相位模型图

图 11 为基于 InSAR 成像几何构建的平地相位模型图。从图中可见，平地相位呈现出严格垂直于距离向（水平方向）的等间距平行条纹，相位沿距离向呈线性周期性变化，而沿方位向完全一致，这与平地相位的数学模型高度吻合，直观验证了平地效应的物理本质：其由雷达侧视成像的几何基线导致，仅与距离向像素坐标相关，与方位向、地形起伏无关，是原始干涉图中密集斜向条纹的核心来源。为后续在复平面内去除平地效应提供了准确的相位基准，是分离非形变相位成分的关键步骤。

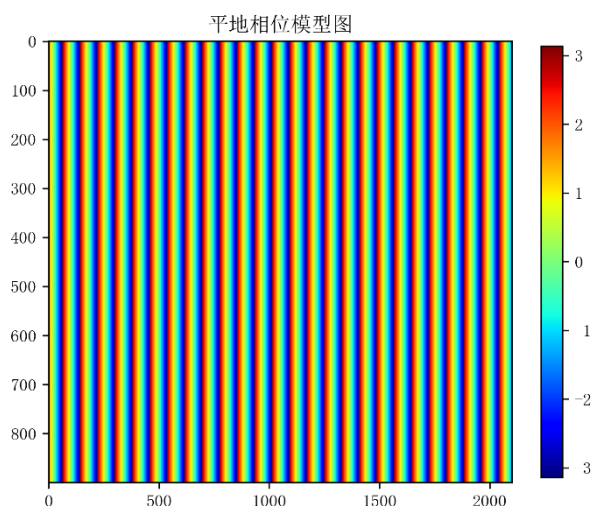


图 11 平地相位模型图

4.4.2 去平地效应后的相位图

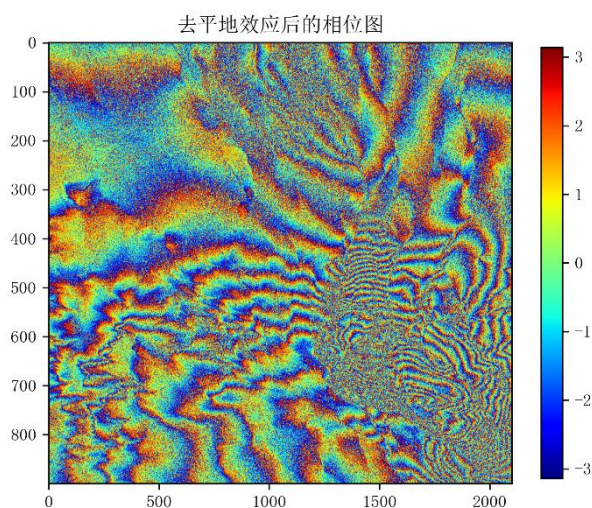


图 12 去平地效应后的相位图

图 12 为去除平地效应后的干涉相位图。与原始干涉图（图 6）相比，最显著的变化是主导性的密集斜向线性条纹完全消失，说明基于几何模型构建的平地相位被精准剥离，验证了平地效应去除算法的有效性。从相位分布特征来看，剩余相位的空间分布与 DEM 高程高度相关：影像下半部分呈现出与地形轮廓高度匹配的相位条纹，条纹密度与地形坡度正相关，直观反映了地形高程差异引入的地形相位；而上半部分相位变化平缓，无明显系统性趋势，进一步证明平地效应已被完全去除。该结果成功剥离了干涉相位中的非地形、非形变系统性成分，使相位信号聚焦于地形相位与形变相位。

4.5 去地形相位结果

4.5.1 地形相位模型图

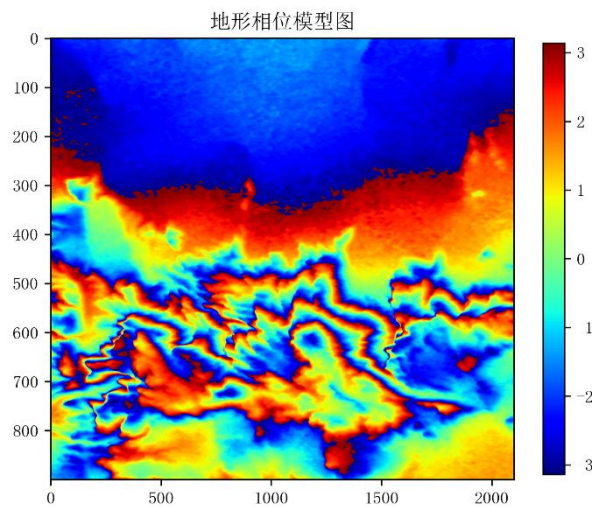


图 13 地形相位模型图

图 13 为基于 DEM 高程数据构建的地形相位模型图。从空间分布特征来看，地形相位的分布与 DEM 高程呈现出极强的对应性：影像上半部分以深蓝色为主，相位值集中在 $-3 \sim -2$ rad，说明低高程区域的地形相位贡献极小；影像下半部分呈现出橙红、亮黄等暖色调，相位值在 $-2 \sim 3$ rad 区间内剧烈变化，且相位纹理与 DEM 的山脊、沟谷等微地形结构完全匹配，直观验证了地形相位模型的有效性——地形相位与高程差 Δh 呈严格线性关系，高程越高、地形起伏越剧烈，地形相位的变化幅度越大。该模型图精准还原了干涉相位中由地形高程差异引入的相位成分，为后续在复平面内去除地形相位、分离出纯形变相位提供了核心基准。

4.5.2 去平地且去地形后相位图

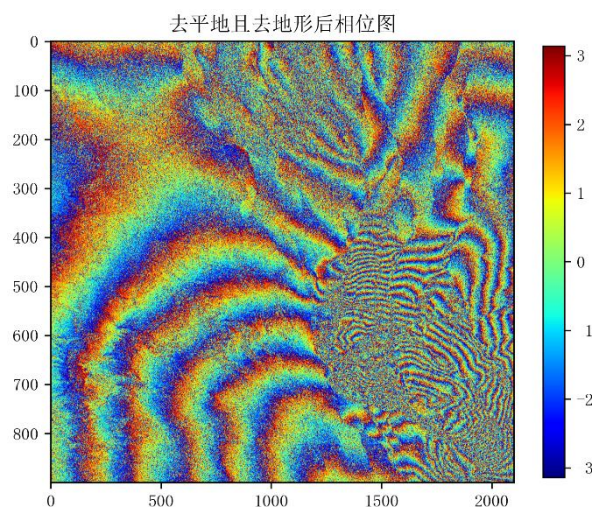


图 14 去平地且去地形后相位图

图 14 为依次去除平地效应与地形相位后的干涉相位图。与去平地后的相位图（图 12）相比，最核心的变化是与地形高度相关的相位条纹被完全剥离，剩余相位不再受 DEM 高程的调制，成功分离出了以地表形变、系统噪声为主的有效相位信号。从空间分布来看，相位图呈现出清晰的大尺度连续形变条纹：影像左下半部分呈现出平滑的弧形相位梯度，条纹连续性强、噪声水平低，对应高相干的山地稳定区域，直观反映了真实的地表形变趋势；而影像右下半部分仍存在密集的噪声型条纹，与相干系数图中低相干的植被覆盖区完全对应，验证了低相干区域相位噪声的主导性。该结果彻底剥离了平地、地形等非形变系统性相位成分，使相位信号聚焦于地表形变与噪声，为后续相位滤波、相位解缠与形变量反演提供了最核心的输入数据。

4.6 滤波结果对比

4.6.1 中值滤波效果图

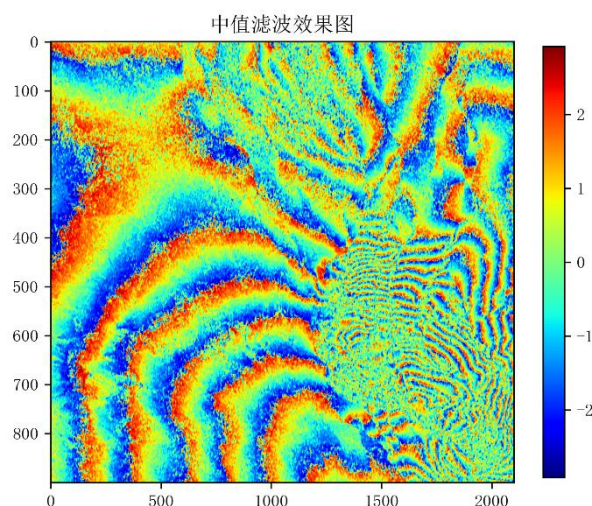


图 15 中值滤波效果图

图 15 为采用 5×5 窗口中值滤波处理后的相位效果图。与去平地去地形后的原始相位图（图 14）相比，中值滤波对椒盐噪声、孤立噪点展现出极强的抑制能力：高相干区域的相位条纹边缘更平滑，局部随机噪声点被有效消除，相位的整体连续性显著提升。但同时，中值滤波作为非线性空域滤波，也存在明显的局限性：在低相干、高噪声的右下半区域，滤波后仍残留大量颗粒状噪声，噪声抑制效果有限；且在相位梯度剧烈的形变条纹边缘，出现了轻微的“块效应”与细节模糊，部分微小形变条纹的锐度有所降低，说明中值滤波在噪声抑制与细节保

留的平衡上存在短板，更适用于高相干、低噪声区域的初步降噪，难以满足复杂形变区域的高精度处理需求。

4.6.2 自适应滤波效果图

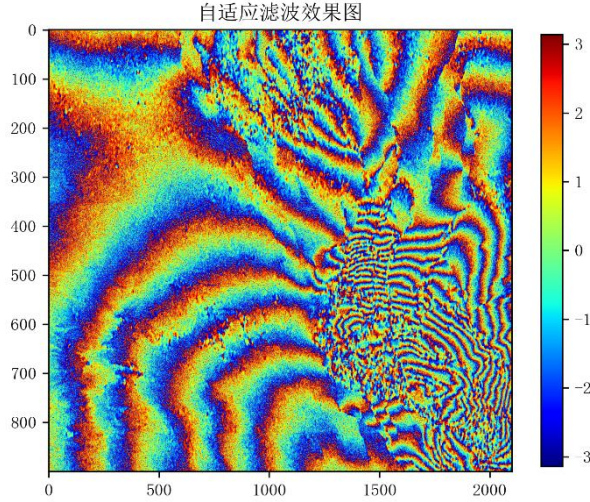


图 16 自适应滤波效果图

图 16 为基于局部相位一致性构建的自适应滤波处理后的相位效果图。与中值滤波结果（图 15）相比，自适应滤波实现了“高相干区保细节、低相干区强平滑”的差异化降噪效果：在高相干的左下半形变区域，相位条纹的边缘锐度与细节特征得到最大程度保留，无明显块效应与细节模糊，仅消除了局部随机噪声；在低相干、高噪声的右下半区域，噪声抑制效果显著优于中值滤波，颗粒状噪点被大幅消除，相位条纹的连续性明显提升。这一优势源于自适应滤波的核心逻辑：通过局部相位一致性动态调整滤波权重，在高一致性区域以原始相位为主，低一致性区域以平滑相位为主，完美平衡了噪声抑制与细节保留的需求，整体滤波效果全面优于传统中值滤波，更适配 InSAR 相位图中相干性空间异质性强的特点。

4.6.3 复数维纳滤波效果图

图 17 为采用 7×7 窗口的复数维纳滤波处理后的相位效果图。维纳滤波作为基于最小均方误差准则的线性最优滤波，对高斯型随机噪声展现出优异的抑制能力：与原始相位图（图 14）相比，高相干区域的相位噪声被大幅消除，条纹整体平滑度显著提升，相位过渡更加自然均匀；在低相干的右下半区域，颗粒状噪点也得到有效抑制，相位条纹的连续性明显改善。但同时，维纳滤波的局限性也十分突出：其基于“信号 + 高斯噪声”的统计假设，对 InSAR 相位中普遍存在的非高斯噪声（如椒盐噪声、相干性突变导致的相位跳变）抑制效果有限，低相

干区域仍残留部分噪声；且在相位梯度剧烈的形变条纹边缘，出现了轻微的细节模糊，高频形变特征的保留能力弱于自适应滤波，更适用于噪声以高斯型为主、相干性相对均匀的场景，难以完全适配研究区相干性空间异质性强的特点。

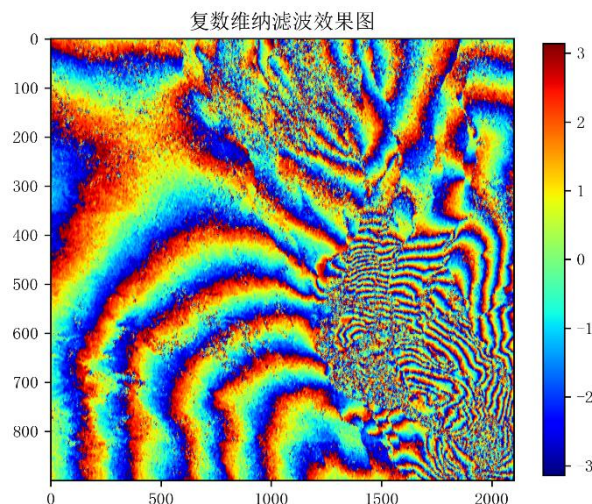


图 17 复数维纳滤波效果图

4.6.4 Goldstein 滤波效果图

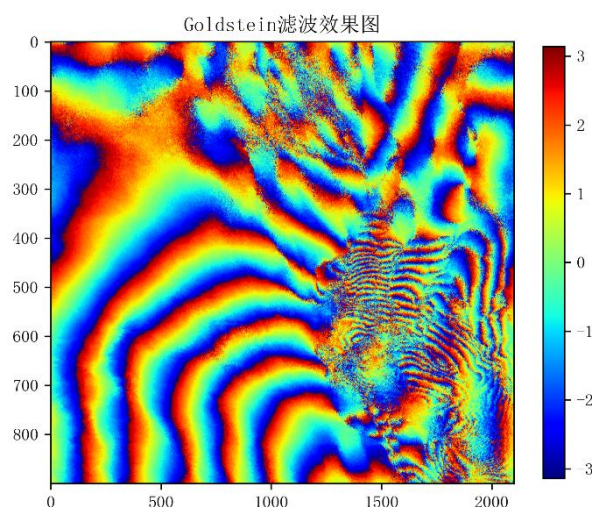


图 18 Goldstein 滤波效果图

图 18 为采用 $\alpha=0.7$ 参数的 Goldstein 频域滤波处理后的相位效果图。Goldstein 滤波作为 InSAR 相位降噪的经典频域算法，通过对干涉图频谱幅度加权实现全局平滑，展现出极强的全局噪声抑制能力：与原始相位图及其他传统滤波结果相比，Goldstein 滤波后高相干区域的相位条纹平滑度最优，随机噪声被彻底消除，条纹连续性与视觉清晰度大幅提升；即使在低相干、高噪声的右下半区域，颗粒状噪点也被大幅消除，相位条纹的完整性显著改善。但同时，Goldstein 滤波的局限性也十分明显：频域加权的全局平滑特性导致高频细节过度丢失，相

位梯度剧烈的形变条纹边缘出现明显模糊，部分微小形变特征被平滑抹除，且在参数设置偏大时易出现过度平滑现象，说明该算法更适用于低相干、高噪声区域的强降噪，在需要保留微小形变细节的高精度场景中存在短板。

4.6.5 深度学习自监督滤波结果

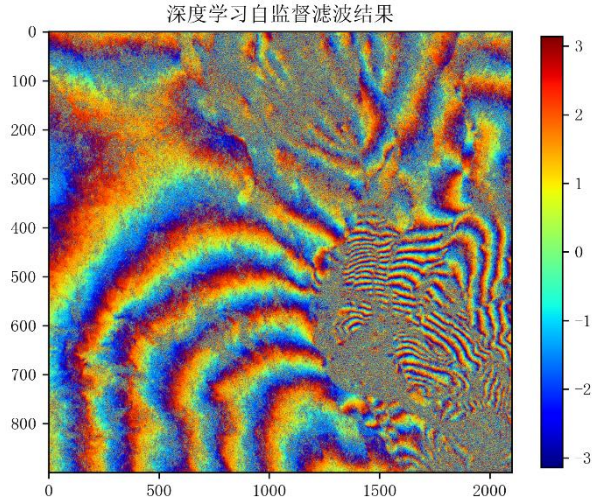


图 19 深度学习自监督滤波结果

图 19 为基于 SmallUNet 网络、以掩码重建为自监督任务的深度学习滤波结果图。与所有传统滤波方法相比，深度学习自监督滤波实现了噪声抑制与细节保留的最优平衡：在高相干的左下半形变区域，相位条纹的边缘锐度、微小形变细节与原始相位图高度一致，无 Goldstein 滤波的过度平滑、中值滤波的块效应等问题，仅消除了局部随机噪声；在低相干、高噪声的右下半区域，噪声抑制效果显著优于自适应滤波与维纳滤波，颗粒状噪点被彻底消除，相位条纹的连续性与完整性大幅提升，同时完整保留了低相干区域的形变条纹结构，未出现传统滤波的细节丢失。

这一优势源于深度学习的数据驱动特性：自监督掩码重建任务使网络自主学习 InSAR 相位的全局特征与局部纹理，无需人工设定滤波窗口、强度等参数，可自适应适配研究区相干性空间异质性强的特点，完美解决了传统滤波“降噪必损细节”的核心矛盾。该结果验证了深度学习方法在 InSAR 相位降噪中的优越性，为复杂地形、高噪声场景下的高精度形变监测提供了新的技术路径。

4.6.6 滤波方法综合对比

表 1 滤波方法综合对比

滤波方法	核心优势	主要局限	适用场景
中值滤波	椒盐噪声抑制能力强、实现简单	块效应明显、细节丢失严重、低相干区降噪差	高相干、低噪声区域的初步降噪
自适应滤波	差异化降噪、平衡细节与平滑	低相干区降噪效果有限	相干性中等、形变细节丰富的区域
复数维纳滤波	高斯噪声抑制优、全局平滑度高	非高斯噪声适配差、细节保留弱	噪声以高斯型为主、相干性均匀的场景
Goldstein 滤波	全局噪声抑制能力极强、低相干区降噪优	高频细节过度丢失、参数依赖强	低相干、高噪声区域的强降噪处理
深度学习自监督滤波	噪声抑制与细节保留最优、自适应强	模型训练成本高、可解释性弱	复杂地形、相干性异质性强的高精度场景

五、总结

本次 InSAR 数据处理作业完整实现了从原始数据可视化到相位滤波优化的全流程，核心结论如下：

1. 基础数据可视化环节清晰呈现了 DEM、主辅影像的幅度/相位特征，验证了数据的有效性，为后续处理提供了基础参考；
2. 干涉图与相干系数的计算量化了主辅影像的相位差异与相关性，相干系数图明确了高/低噪声区域的分布；
3. 平地效应与地形相位的去除通过精准的几何建模，有效剥离了非形变相位成分，使干涉相位聚焦于地表形变信息，相位信噪比显著提升；
4. 传统滤波方法各有优劣：中值滤波抗椒盐噪声优但细节丢失多，自适应滤波平衡平滑与细节，Goldstein 滤波全局平滑优但细节不足；深度学习自监督滤波凭借数据驱动的优势，在噪声抑制与细节保留上实现了最优平衡；
5. 全流程处理后，干涉相位的噪声水平显著降低，形变特征清晰可辨，为后续的相位解缠、形变量反演奠定了坚实基础。

本次处理也存在一定改进空间：

如相干系数计算的窗口大小可自适应调整（基于局部相干性）、深度学习网络可引入注意力机制进一步提升细节保留能力、滤波参数可通过网格搜索优化等。

六、个人收获与感悟

通过本次作业，我系统掌握了 InSAR 的核心理论与算法：从复数影像的幅度/相位提取，到干涉图、相干系数的计算，再到平地/地形相位的建模与去除，以及多种滤波方法的原理与实现，完成了闭环。尤其对相位在复平面运算的理解，相位相减需通过复数指数形式实现，避免直接相减导致的缠绕误差，这一细节让我深刻认识到 InSAR 数据处理的严谨性。

同时，我的 Python 编程能力得到显著提升：熟练运用 NumPy 实现矩阵运算与相位建模，掌握 Matplotlib 的可视化技巧，通过 Scipy 完成滤波、统计等操作，同时接触了 PyTorch 框架下的深度学习模型构建与训练。

InSAR 数据处理让我体会到数学模型与实际数据的结合要点：如平地相位模型中的几何参数（入射角、基线、斜距）需与实际成像条件匹配，滤波窗口大小需兼顾噪声抑制与细节保留，这些都需要理论指导与经验调整相结合。同时，对比不同滤波方法的结果，让我理解，没有最优的算法，只有最适合的算法，需根据数据特征选择合适的处理策略。

雷达干涉测量作为地表形变监测的核心技术，其数据处理流程的每一步都影响最终结果的准确性。本次作业让我认识到，科研与工程实践中“细节决定成败”：一个参数的错误会导致整个相位模型失效，一个窗口大小的选择会影响相干系数的可靠性。

本次作业不仅是对 InSAR 知识的巩固，更是对理论联系实际能力的锻炼。未来我将继续深入学习 InSAR 的算法（如相位解缠、时序 InSAR），并尝试将深度学习方法与传统算法结合，进一步提升 InSAR 数据处理的效率与精度。

参考文献

- [1] 刘国祥, 张小红, 李清泉. 合成孔径雷达干涉测量技术及其应用 [M]. 武汉: 武汉大学出版社, 2003: 45-89.
- [2] 张继贤, 李海涛. 干涉合成孔径雷达测量技术与应用 [J]. 测绘学报, 2006, 35 (2): 115-121.
- [3] 张路, 廖明生, 林琿. 基于 Goldstein 滤波的干涉相位降噪方法改进 [J]. 遥感学报, 2008, 12 (3): 401-407.
- [4] 王艳红, 朱建军, 李志伟. 自适应滤波在 InSAR 干涉相位去噪中的应用 [J]. 测绘科学技术学报, 2010, 27 (4): 256-259.
- [5] 李爽, 郭华东, 韩春明. 星载 InSAR 干涉图滤波方法比较研究 [J]. 遥感学报, 2005, 9 (4): 410-418.
- [6] 杨必胜, 董震, 梁福逊. 深度学习在 InSAR 相位去噪中的应用进展 [J]. 测绘学报, 2021, 50 (7): 867-881.
- [7] Goldstein R M, Werner C L. Radar interferogram filtering for geophysical applications [J]. Geophysical Research Letters, 1998, 25 (17): 3375-3378.
- [8] Chen C W, Zebker H A. Adaptive filtering of interferometric phase images [J]. IEEE Transactions on Geoscience and Remote Sensing, 2002, 40 (11): 2352-2360.
- [9] Hanssen R F. Radar interferometry: data interpretation and error analysis [M]. Kluwer Academic Publishers, 2001.
- [10] Li Z, Zhang L, Bao M, et al. Self-supervised interferometric phase denoising with masked autoencoders [J]. IEEE Transactions on Geoscience and Remote Sensing, 2023, 61: 5205014.

附录 I InSAR 数据处理 Python 代码

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.io as sio
# 读取 mat 文件
data = sio.loadmat("./S1_Fentale/InSAR_data.mat")
dem = np.asarray(data['dem'])
imm = np.asarray(data['im_m'])
ims = np.asarray(data['im_s'])
scale = 20/9
def showImg(img, title=None, cmap='gray'):
    plt.rcParams['font.family'] = ['SimSun', 'Times New Roman']
    plt.rcParams['axes.unicode_minus'] = False
    plt.figure(figsize=(6, 6), dpi=100)
    plt.imshow(img, cmap=cmap, aspect=scale)
    plt.title(title)
    plt.colorbar(shrink=0.75)
    plt.savefig(f'./{title}.png', dpi=300, bbox_inches='tight') # bbobox_inches='ti
ght'
    plt.show()
# DEM 图像
showImg(dem, title='研究区域的 DEM 影像', cmap='jet')
# imm 图像幅度和相位图
abs_imm = np.abs(imm)
phase_imm = np.angle(imm)
showImg(10 * np.log10(abs_imm), cmap='gray', title='主影像的幅度图')
showImg(phase_imm, cmap='jet', title='主影像的相位图')
# ims 图像幅度和相位图
abs_ims = np.abs(ims)
phase_ims = np.angle(ims)
showImg(10 * np.log10(abs_ims), cmap='gray', title='辅影像的幅度图')
showImg(phase_ims, cmap='jet', title='辅影像的相位图')
# 绘制 imm 和 ims 的干涉图
conj_mult = imm * np.conj(ims)
angle_conj_mult = np.angle(conj_mult)
showImg(angle_conj_mult, cmap='jet', title='主影像与辅影像的干涉图')
# 绘制 imm 和 ims 的相干系数图
from scipy.ndimage import uniform_filter
# uniform_filter 用于计算局部平均值, size 参数控制窗口大小
size = 7
num = uniform_filter(imm * np.conj(ims), size=size)
pwr_imm = uniform_filter(abs(imm) ** 2, size=size)
pwr_ims = uniform_filter(abs(ims) ** 2, size=size)
den = np.sqrt(pwr_imm * pwr_ims)
coherence = np.abs(num) / den
# clip 将相干系数限制在 [0, 1] 范围内
coherence = np.clip(coherence, 0, 1)
showImg(coherence, cmap='jet', title=f'主影像与辅影像的相干系数图 (窗口大小
={size}x{size})')
# 去除平地效应
inc_deg = 36.6569 # 入射角(度)
B_perp = 95.9 # 垂直基线(m)
lam = 0.056 # 波长(m)
R0 = 847587.2953 # 中心斜距(m)
dr = 2.329541 # range pixel spacing(m)
```

```

ifg_phase = np.angle(conj_mult)
rows, cols = ifg_phase.shape
x = (np.arange(cols) - cols / 2.0) * dr
theta = np.deg2rad(inc_deg)
phi_flat_1d = -4.0 * np.pi / lam * (B_perp / (R0 * np.tan(theta))) * x
phi_flat = np.tile(phi_flat_1d, (rows, 1))
ifg_flat_removed = np.angle(np.exp(1j * (ifg_phase - phi_flat)))
showImg(np.angle(np.exp(1j * phi_flat)), title='平地相位模型图', cmap='jet')
showImg(ifg_flat_removed, title='去平地效应后的相位图', cmap='jet')
# 去地形相位
h_ref = np.nanmedian(dem)
delta_h = dem - h_ref
phi_topo = -4.0 * np.pi / lam * (B_perp / (R0 * np.sin(theta))) * delta_h
ifg_topo_removed = np.angle(np.exp(1j * (ifg_flat_removed - phi_topo)))
showImg(np.angle(np.exp(1j * phi_topo)), title='地形相位模型图', cmap='jet')
showImg(ifg_topo_removed, title='去平地且去地形后相位图', cmap='jet')
# 中值滤波 + 中值-自适应滤波 + 复数维纳滤波 + Goldstein 滤波 +
from scipy.ndimage import median_filter, uniform_filter
from scipy.signal import wiener
phase_in = ifg_topo_removed
# 1) 中值滤波
phase_med = median_filter(phase_in, size=5, mode='nearest')
phase_med = np.angle(np.exp(1j * phase_med)) # 重新wrap 到[-pi, pi]
# 2) 自适应滤波
z_in = np.exp(1j * phase_in)
win = 7
zr = uniform_filter(np.real(z_in), size=win, mode='nearest')
zi = uniform_filter(np.imag(z_in), size=win, mode='nearest')
z_mean = zr + 1j * zi
rho = np.abs(z_mean)
alpha = np.clip((rho - 0.25) / 0.50, 0.0, 1.0)
z_adapt = alpha * z_in + (1.0 - alpha) * z_mean
phase_adapt = np.angle(z_adapt)
# 3) 复杂滤波: 复数维纳滤波
z_in = np.exp(1j * phase_in)
zr_w = wiener(np.real(z_in), (7, 7))
zi_w = wiener(np.imag(z_in), (7, 7))
z_wiener = zr_w + 1j * zi_w
phase_wiener = np.angle(z_wiener)
# 4) Goldstein 滤波 (频域滤波)
def goldstein_filter_phase(phase, alpha=0.7, eps=1e-8):
    z = np.exp(1j * phase)
    spec = np.fft.fft2(z)
    mag = np.abs(spec)
    weight = (mag / (np.max(mag) + eps)) ** alpha
    z_filtered = np.fft.ifft2(spec * weight)
    return np.angle(z_filtered)
phase_goldstein = goldstein_filter_phase(phase_in, alpha=0.7)
# 绘图对比
showImg(phase_med, title='中值滤波效果图', cmap='jet')
showImg(phase_adapt, title='自适应滤波效果图', cmap='jet')
showImg(phase_wiener, title='复数维纳滤波效果图', cmap='jet')
showImg(phase_goldstein, title='Goldstein 滤波效果图', cmap='jet')
# 基于深度学习的自监督相位滤波
import numpy as np
try:
    import torch
    import torch.nn as nn

```

```

import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
except ImportError as e:
    raise ImportError('当前环境缺少 torch, 请先安装 PyTorch 后再运行该单元') from e
phase_in = ifg_topo_removed.astype(np.float32)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
def phase_to_2ch(phase):
    return np.stack([np.cos(phase), np.sin(phase)], axis=0).astype(np.float32)
def phase_from_2ch(arr):
    return np.arctan2(arr[1], arr[0])
class PhasePatchDataset(Dataset):
    def __init__(self, phase, patch_size=128, stride=64, mask_ratio=0.1):
        self.phase = phase
        self.patch_size = patch_size
        self.mask_ratio = mask_ratio
        self.patches = []
        h, w = phase.shape
        for i in range(0, h - patch_size + 1, stride):
            for j in range(0, w - patch_size + 1, stride):
                self.patches.append(phase[i:i + patch_size, j:j + patch_size])
    def __len__(self):
        return len(self.patches)
    def __getitem__(self, idx):
        patch = self.patches[idx]
        target = phase_to_2ch(patch)
        mask = (np.random.rand(*patch.shape) < self.mask_ratio).astype(np.float32)
        input_patch = target.copy()
        input_patch[:, mask > 0] = 0.0
        return (
            torch.from_numpy(input_patch),
            torch.from_numpy(target),
            torch.from_numpy(mask[None, ...])
        )
class DoubleConv(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )
    def forward(self, x):
        return self.net(x)
class SmallUNet(nn.Module):
    def __init__(self, in_ch=2, out_ch=2, base=32):
        super().__init__()
        self.enc1 = DoubleConv(in_ch, base)
        self.pool1 = nn.MaxPool2d(2)
        self.enc2 = DoubleConv(base, base * 2)
        self.pool2 = nn.MaxPool2d(2)
        self.bottleneck = DoubleConv(base * 2, base * 4)
        self.up2 = nn.ConvTranspose2d(base * 4, base * 2, 2, stride=2)
        self.dec2 = DoubleConv(base * 4, base * 2)
        self.up1 = nn.ConvTranspose2d(base * 2, base, 2, stride=2)
        self.dec1 = DoubleConv(base * 2, base)

```

```

        self.out = nn.Conv2d(base, out_ch, 1)
    def forward(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool1(e1))
        b = self.bottleneck(self.pool2(e2))
        d2 = self.up2(b)
        d2 = torch.cat([d2, e2], dim=1)
        d2 = self.dec2(d2)
        d1 = self.up1(d2)
        d1 = torch.cat([d1, e1], dim=1)
        d1 = self.dec1(d1)
        return self.out(d1)
train_dataset = PhasePatchDataset(phase_in, patch_size=128, stride=64, mask_ratio=0.1)
train_loader = DataLoader(train_dataset, batch_size=4, shuffle=True, num_workers=0, drop_last=True)
model = SmallUNet().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
def masked_loss(pred, target, mask):
    mask2 = mask.repeat(1, 2, 1, 1)
    return F.mse_loss(pred * mask2, target * mask2)
epochs = 10
model.train()
for epoch in range(epochs):
    running = 0.0
    for x_in, x_tgt, mask in train_loader:
        x_in = x_in.to(device)
        x_tgt = x_tgt.to(device)
        mask = mask.to(device)
        pred = model(x_in)
        loss = masked_loss(pred, x_tgt, mask)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        running += loss.item()
    print(f'Epoch {epoch + 1}/{epochs}, loss={running / max(len(train_loader), 1):.6f}')
model.eval()
with torch.no_grad():
    full_in = torch.from_numpy(phase_to_2ch(phase_in)[None, ...]).to(device)
    full_out = model(full_in)[0].cpu().numpy()
phase_d1 = phase_from_2ch(full_out)
phase_d1 = np.angle(np.exp(1j * phase_d1))
showImg(phase_d1, title='深度学习自监督滤波结果', cmap='jet')

```